


INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

# *Proyecto MiBici: MLflow Project y tracking avanzado*

Equipo 34

Diego Falcón Costilla A01139580

Oscar Elton A01840474

Mariana Favarony Avila A01704671

Javier Alejandro Garcia Flores A01797672

Operaciones de aprendizaje automático (TC5061)

## 1. Repositorio y reproducción

Repositorio del equipo: github.com/diegofalconcostilla/TC5061-Proyecto-Equipo34. Esta entrega continúa la propuesta de la semana anterior: el mismo repositorio, el mismo experimento `mibici-equipo34`, los mismos datos (`data_version = efa3a16661ec`), la misma partición temporal (entrenamiento 2024-01 a 2025-12, validación 2026-01 a 2026-04 y prueba 2026-05 a 2026-08) y las mismas métricas. El repositorio ahora es un MLflow Project: todas las corridas de este reporte se lanzaron con `mlflow run` desde un clon limpio, contra un servidor de tracking vacío respaldado por SQLite. Las 64 corridas de la evidencia tienen el mismo commit y `git_dirty = false`.

Comandos para reproducir los resultados desde un clon limpio (detalle en el README).

```
git clone https://github.com/diegofalconcostilla/TC5061-Proyecto-Equipo34.git
cd TC5061-Proyecto-Equipo34
# Terminal 1: servidor de tracking con base de datos (necesario para el Model
Registry)
mlflow server --backend-store-uri sqlite:///mlflow.db --port 5001
# Terminal 2
export MLFLOW_TRACKING_URI=http://127.0.0.1:5001
mlflow run . -e ingest --experiment-name mibici-equipo34     # descarga, limpia y
versiona los datos
bash scripts/run_experiments.sh                              # 57 corridas y 6
registros (= make all)

# Modelo elegido de cada pregunta
mlflow run . -e train -P question=p1 -P model=hgb -P params="learning_rate=0.1" --
experiment-name mibici-equipo34
mlflow run . -e train -P question=p2 -P model=hgb -P params="learning_rate=0.1" --
experiment-name mibici-equipo34
mlflow run . -e train -P question=p3 -P model=kmeans --experiment-name mibici-equipo34
# Registro y alias: mlflow run . -e register -P question=p3 -P run_name="p3-kmeans-
perfiles-uso" -P alias=champion
```

El servidor usa el puerto 5001 porque en macOS el 5000 lo ocupa AirPlay. `mlflow.db`, `mlruns/`, `mlartifacts/`, los datos y los modelos serializados están en `.gitignore` y no aparecen en el historial de Git.

## 2. MLflow Project

El archivo `MLproject` en la raíz sigue el patrón de pipeline reproducible y versionado (Lakshmanan et al., 2020; MLflow, s.f.-b): declara el entorno `python_env.yaml` (Python 3.12 e instalación de `requirements.txt`, donde todas las dependencias tienen versión fija) y cuatro puntos de entrada. Rutas, fechas, cortes, semilla e hiperparámetros por defecto no se repiten en el `MLproject`: viven en `mibici/config.py`.

Tabla 1. Puntos de entrada del MLflow Project.

| Punto de entrada | Parámetros | Qué hace |
|---|---|---|
| `ingest` | ninguno | Ejecuta `dataset.py` (descarga, limpieza, eliminación de datos personales y manifiesto), construye los datasets de P1 a P3 con `features.py` y registra el manifiesto en una corrida `ingest-<data_version>`. |
| `train` | `question` (p1, p2, p3), `model`, `params` ("k=v,k2=v2"), `stage` | Entrena un modelo de la tabla de su pregunta con los hiperparámetros de `config.py` más los cambios de `params`; valida tipos y rechaza nombres desconocidos. |
| `search` | `question`, `model`, `grid` ("k=v1\|v2;k2=v3\|v4"), `params` | Búsqueda de hiperparámetros con nested runs: una corrida padre y una hija por combinación; el padre elige la mejor por la métrica de validación y guarda `search/results.csv`. |
| `register` | `question`, `run_name`, `alias` | Registra el modelo de una corrida en el Model Registry con descripción y linaje, le asigna el alias y verifica que `models:/<nombre>@<alias>` cargue y prediga. |

En esta entrega se corrigió un problema que impedía reproducir el proyecto en Windows: `mlflow run` pasa un parámetro vacío como `""` literal y los puntos de entrada lo rechazaban. Además, `scripts/run_experiments.sh` llama a `mlflow run` directamente, así que funciona sin `make`.

## 3. Diseño de la experimentación

**Corridas.** El experimento tiene 64 corridas: la ingesta, 19 entrenamientos individuales, seis búsquedas (seis corridas padre con 32 corridas hijas) y seis corridas de registro. Por pregunta hay los baselines de la entrega anterior y al menos cinco versiones adicionales que varían el algoritmo, los hiperparámetros o ambos. Las variantes RandomForest, Ridge, MiniBatchKMeans, Birch y mezcla de gaussianas, que se habían propuesto en el PR #15, se integraron a las tablas de modelos para que registren los mismos artefactos que el resto.

**Comparabilidad.** Todas las corridas usan la partición temporal y las métricas de la entrega anterior, y el modelo se elige siempre con la métrica de validación; prueba solo se reporta. Las métricas de selección son las ligadas a la decisión de cada usuario: WAPE de validación en P1 (error relativo agregado, que se traduce en bicicletas mal ubicadas), PR-AUC de validación en P2 (ordenar bien las estaciones en riesgo cuando solo 2.5 % de los bloques lo están; con clases tan desbalanceadas es más informativa que el ROC-AUC, según Saito y Rehmsmeier, 2015) y silueta (Rousseeuw, 1987) en el espacio de perfiles de uso en P3, que se acompaña del ARI de estabilidad entre el periodo de entrenamiento y el de prueba.

**Etiquetas.** Siguiendo la API de tracking (MLflow, s.f.-c), cada corrida lleva `question`, `model_family`, `stage_of_work` (`baseline`, `candidate`, `search`, `ingest`, `register`) y `model_role`, además de las obligatorias `team`, `git_commit`, `git_branch`, `git_dirty` y `data_version`. Con ellas la interfaz filtra, por ejemplo, `tags.question = 'P2' and tags.stage_of_work != 'baseline'`.

**Artefactos.** Cada corrida con modelo guarda, en subcarpetas:

- **Modelo**: con `mlflow.sklearn.log_model`, su firma (`signature`) y un ejemplo de entrada. En MLflow 3 el modelo se guarda como *Logged Model* ligado a la corrida (Figura A3c). En P2 se registra el clasificador junto con el umbral de decisión elegido en validación, para que el modelo del registry decida igual que en el experimento.
- `data/`: el `manifest.json` de la ingesta (versión de los datos) y `dataset.json` con ruta, hash y tamaño del conjunto de entrenamiento, que además se registra con `mlflow.log_input`.
- `plots/`: serie diaria y real contra predicho con residuales (P1), curva precisión-exhaustividad y matriz de confusión (P2), mapa y perfiles de los grupos y curva de selección de k (P3).
- `config/`: `run_config.json` con los hiperparámetros resueltos y una copia de `config.py`; `metrics/`: las métricas en JSON y, en P3, la asignación de estaciones a grupos.

Ningún artefacto contiene datos personales: la ingesta elimina `Usuario_Id`, `Genero` y `Año_de_nacimiento` antes de escribir cualquier archivo, y los artefactos solo contienen agregados por estación y los identificadores y coordenadas públicos de las estaciones.

## 4. Comparación de experimentos

Las Tablas 2 a 4 muestran, por pregunta, los baselines, las candidatas y las corridas hijas de las búsquedas. La mejora se calcula contra el baseline registrado en la entrega anterior (naive estacional en P1, persistencia semanal en P2 y KMeans geográfico en P3). En negritas, el modelo que pasó a producción (`champion`). Las cifras salen de la corrida oficial y cualquiera las puede consultar en la interfaz de MLflow filtrando por `tags.question`.

Tabla 2. P1, viajes por estación y día (selección: WAPE de validación, menor es mejor).

| Tipo | Modelo | Hiperparámetros clave | Valid MAE | Valid WAPE | Test MAE | Test WAPE | Test $R^2$ | Mejora WAPE test |
|---|---|---|---|---|---|---|---|---|
| baseline | Naive estacional 7 d | — | 7.91 | 0.258 | 7.12 | 0.261 | 0.794 | +0.0 % |
| baseline | Media móvil 7 d | — | 8.27 | 0.270 | 7.14 | 0.262 | 0.838 | -0.3 % |
| baseline | Promedio histórico estación-día | — | 7.42 | 0.242 | 8.11 | 0.298 | 0.711 | -13.9 % |
| candidata | Poisson lineal | alpha=0.0001 | 6.07 | 0.198 | 6.00 | 0.220 | 0.862 | +15.8 % |
| candidata | Ridge | alpha=1.0 | 6.57 | 0.214 | 6.36 | 0.233 | 0.873 | +10.7 % |
| candidata | Random Forest (challenger) | n_estimators=100, max_depth=15, min_samples_leaf=10 | 5.52 | 0.180 | 5.40 | 0.198 | 0.888 | +24.2 % |
| candidata | HGB | loss=poisson, learning_rate=0.05, max_leaf_nodes=63 | 5.42 | 0.177 | 5.30 | 0.194 | 0.893 | +25.6 % |
| candidata | HGB | loss=squared_error, learning_rate=0.05, max_leaf_nodes=63 | 5.50 | 0.179 | 5.41 | 0.198 | 0.887 | +24.1 % |
| búsqueda | Poisson lineal | alpha=0.0001 | 6.07 | 0.198 | 6.00 | 0.220 | 0.862 | +15.8 % |
| búsqueda | Poisson lineal | alpha=0.01 | 6.07 | 0.198 | 5.99 | 0.220 | 0.862 | +15.9 % |
| búsqueda | Poisson lineal | alpha=0.1 | 6.05 | 0.197 | 5.97 | 0.219 | 0.864 | +16.2 % |
| búsqueda | HGB | loss=poisson, learning_rate=0.03, max_leaf_nodes=31 | 5.48 | 0.179 | 5.28 | 0.194 | 0.897 | +25.9 % |
| búsqueda | HGB | loss=poisson, learning_rate=0.03, max_leaf_nodes=63 | 5.43 | 0.177 | 5.29 | 0.194 | 0.896 | +25.7 % |
| búsqueda | HGB | loss=poisson, learning_rate=0.1, max_leaf_nodes=31 | 5.41 | 0.177 | 5.32 | 0.195 | 0.892 | +25.4 % |
| **búsqueda** | **HGB (champion)** | **loss=poisson, learning_rate=0.1, max_leaf_nodes=63** | **5.41** | **0.176** | **5.33** | **0.196** | **0.891** | **+25.1 %** |

HGB = HistGradientBoosting. WAPE = Σ|real − predicho| / Σ real. La mejora es la reducción relativa del WAPE de prueba frente al naive estacional de 7 días.

Tabla 3. P2, riesgo de vaciado o llenado en el siguiente bloque de 3 h (selección: PR-AUC de validación, mayor es mejor).

| Tipo | Modelo | Hiperparámetros clave | Valid PR-AUC | Valid F1 | Test PR-AUC | Test F1 | Test precisión | Test recall | Mejora PR-AUC test |
|---|---|---|---|---|---|---|---|---|---|
| baseline | Persistencia semanal | — | 0.397 | 0.339 | 0.388 | 0.350 | 0.349 | 0.350 | +0.0 % |
| baseline | Tasa histórica | — | 0.376 | 0.381 | 0.315 | 0.335 | 0.273 | 0.434 | -18.8 % |
| candidata | Logística | C=1.0 | 0.405 | 0.409 | 0.405 | 0.414 | 0.388 | 0.445 | +4.5 % |
| candidata | Random Forest (challenger) | n_estimators=150, max_depth=12, min_samples_leaf=50 | 0.452 | 0.434 | 0.443 | 0.431 | 0.411 | 0.454 | +14.2 % |
| candidata | HGB | learning_rate=0.05, max_leaf_nodes=63 | 0.483 | 0.456 | 0.464 | 0.445 | 0.473 | 0.421 | +19.8 % |
| búsqueda | Logística | C=0.1 | 0.405 | 0.409 | 0.405 | 0.414 | 0.388 | 0.444 | +4.5 % |
| búsqueda | Logística | C=1.0 | 0.405 | 0.409 | 0.405 | 0.414 | 0.388 | 0.445 | +4.5 % |
| búsqueda | Logística | C=10.0 | 0.405 | 0.409 | 0.405 | 0.414 | 0.388 | 0.445 | +4.5 % |
| búsqueda | HGB | learning_rate=0.03, max_leaf_nodes=31 | 0.476 | 0.453 | 0.460 | 0.444 | 0.445 | 0.443 | +18.8 % |
| búsqueda | HGB | learning_rate=0.03, max_leaf_nodes=63 | 0.481 | 0.456 | 0.463 | 0.443 | 0.491 | 0.404 | +19.6 % |
| búsqueda | HGB | learning_rate=0.1, max_leaf_nodes=31 | 0.482 | 0.458 | 0.464 | 0.446 | 0.439 | 0.453 | +19.6 % |
| **búsqueda** | **HGB (champion)** | **learning_rate=0.1, max_leaf_nodes=63** | **0.483** | **0.459** | **0.462** | **0.442** | **0.430** | **0.454** | **+19.1 %** |

HGB = HistGradientBoostingClassifier. Precisión, recall y F1 usan el umbral que maximiza F1 en validación, aplicado sin cambios en prueba. Prevalencia de riesgo en prueba: 2.5 %. La mejora es el aumento relativo del PR-AUC de prueba frente a la persistencia semanal.

Tabla 4. P3, grupos de estaciones por perfil de uso (selección: silueta en el espacio de perfiles; se exige también estabilidad).

| Tipo | Modelo | Hiperparámetros clave | Silueta perfiles | Davies-Bouldin | ARI estabilidad | Grupo mínimo | Mejora silueta |
|---|---|---|---|---|---|---|---|
| baseline | KMeans | n_clusters=6, n_init=20 | -0.014 | 4.48 | 1.000* | 33 | +0.000 |
| baseline | Regla mañana/tarde | — | 0.051 | 3.17 | 0.271 | 43 | +0.064 |
| **candidata** | **KMeans (champion)** | **k=3, n_init=20** | **0.142** | **2.03** | **0.446** | **28** | **+0.156** |
| candidata | MiniBatch KMeans | n_clusters=3, batch_size=1024 | 0.086 | 2.67 | 0.470 | 57 | +0.100 |
| candidata | Birch (challenger) | n_clusters=3, threshold=0.5 | 0.122 | 2.05 | 0.532 | 29 | +0.136 |
| candidata | Mezcla gaussiana | n_clusters=3, covariance_type=full | 0.155 | 2.61 | 0.011 | 23 | +0.169 |
| búsqueda | KMeans | k=3, n_init=20 | 0.142 | 2.03 | 0.446 | 28 | +0.156 |
| búsqueda | KMeans | k=4, n_init=20 | 0.095 | 2.31 | 0.389 | 30 | +0.109 |
| búsqueda | KMeans | k=5, n_init=20 | 0.086 | 2.35 | 0.346 | 30 | +0.100 |
| búsqueda | KMeans | k=8, n_init=20 | 0.071 | 2.26 | 0.208 | 8 | +0.085 |
| búsqueda | Mezcla gaussiana | n_clusters=3, covariance_type=diag | 0.160 | 2.22 | 0.305 | 32 | +0.174 |
| búsqueda | Mezcla gaussiana | n_clusters=4, covariance_type=full | 0.085 | 2.56 | 0.001 | 23 | +0.099 |
| búsqueda | Mezcla gaussiana | n_clusters=4, covariance_type=diag | 0.104 | 2.49 | 0.289 | 23 | +0.118 |
| búsqueda | Mezcla gaussiana | n_clusters=5, covariance_type=full | 0.089 | 2.63 | 0.007 | 30 | +0.103 |
| búsqueda | Mezcla gaussiana | n_clusters=5, covariance_type=diag | 0.115 | 2.38 | 0.297 | 13 | +0.129 |

Silueta y Davies-Bouldin se miden en el mismo espacio estandarizado de perfiles de uso para todos los modelos. ARI de estabilidad: concordancia entre el grupo de cada estación con su perfil de entrenamiento (2024-2025) y con su perfil del periodo de prueba (2026). *El ARI del baseline geográfico es 1 por construcción, porque las coordenadas no cambian. Se muestran las búsquedas de KMeans con n_init = 20 y k = 3, 4, 5 y 8, y la búsqueda completa de la mezcla gaussiana.

## 5. Análisis y selección del modelo

### P1: demanda diaria por estación

**Champion: HistGradientBoosting con pérdida Poisson,** `learning_rate = 0.1` (hijo de la búsqueda `p1-hgb-busqueda`). Reduce el WAPE de prueba de 0.261 a 0.196 (25.1 %) y el MAE de 7.12 a 5.33 viajes por estación-día. Para el operador, eso significa planear la flota de la mañana con un error agregado de una de cada cinco bicicletas en lugar de una de cada cuatro.

- **Desempeño.** Se eligió por el WAPE de validación (0.1764), el más bajo de las 15 corridas. Todas las candidatas superan a los tres baselines. El orden por familias es claro y se repite en validación y en prueba: los modelos de árboles (0.176 a 0.180 de WAPE de validación)

superan a los lineales (0.197 a 0.214), y entre los lineales el enlace logarítmico de Poisson supera a Ridge, porque la demanda es un conteo no negativo con varianza que crece con el nivel. La pérdida Poisson también supera a `squared_error` dentro del HGB (0.1767 frente a 0.1794).

- **Estabilidad.** Las cinco variantes del HGB con pérdida Poisson quedan en un rango de 0.0022 de WAPE de validación y de 0.0020 en prueba. El resultado no depende de un ajuste fino. En prueba la mejor variante es otra (`learning_rate = 0.03`, `max_leaf_nodes = 31`, WAPE 0.1935), pero la diferencia de 0.002 está dentro de esa variación y elegir con prueba sería sobreajustar al periodo de evaluación.
- **Costo.** El champion entrenó en 12 s, la mitad que el HGB con los valores por defecto (23 s), gracias a una tasa de aprendizaje mayor con *early stopping*. El Random Forest tarda 144 s, su modelo serializado es mucho más grande y su WAPE es peor (0.180). El HGB predice las 473 estaciones en milisegundos, así que el costo de inferencia no limita la decisión.
- **Challenger: Random Forest**, la mejor alternativa de otra familia (bagging). Si el boosting empieza a degradarse con datos nuevos, es la opción de respaldo que menos depende de la misma familia de modelos.

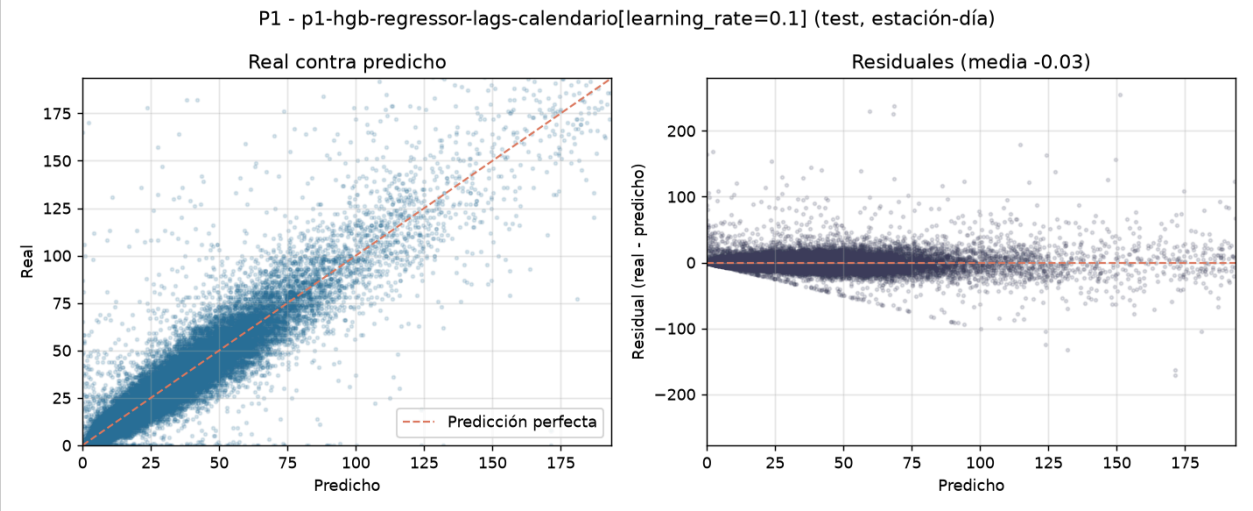



Figura 1. P1, champion en prueba: real contra predicho y residuales por estación-día (artefacto `plots/` de la corrida de origen).

**P2: riesgo de desbalance por bloque de 3 horas**

**Champion: HistGradientBoosting balanceado,** **`learning_rate = 0.1`** (hijo de `p2-hgb-busqueda`). Eleva el PR-AUC de prueba de 0.388 a 0.462 (19.1 %) con una prevalencia de 2.5 %: un clasificador aleatorio tendría 0.025. Con el umbral de validación detecta 45.4 % de los bloques en riesgo con una precisión de 43 %. En la lista de estaciones que recibe el equipo de rebalanceo, cerca de dos de cada cinco visitas evitan un vaciado o llenado real, frente a una de cada tres con la persistencia semanal.

- **Desempeño.** PR-AUC de validación de 0.4827, el más alto. El orden es otra vez consistente en validación y en prueba: HGB (0.476 a 0.483), Random Forest (0.452), logística (0.405) y baselines (0.376 a 0.397). La logística no responde a la regularización (C de 0.1 a 10 da el mismo PR-AUC hasta la cuarta cifra): su límite es la forma lineal, no el sobreajuste, y por eso un modelo de árboles es necesario.
- **Estabilidad.** Las cinco variantes del HGB difieren en menos de 0.007 de PR-AUC de validación y en menos de 0.004 en prueba. El champion queda 0.003 por debajo del HGB por defecto en prueba, lo que confirma que las variantes son equivalentes. Lo que sí cambia entre variantes es el equilibrio entre precisión y recall: con `learning_rate = 0.03` la precisión sube a 0.49 pero el recall baja a 0.40. El champion tiene el recall más alto de la familia (0.454), que es lo que importa cuando dejar pasar un bloque en riesgo cuesta más que una visita de más.
- **Costo.** 51 s de entrenamiento frente a 81 s del HGB por defecto y 83 s del Random Forest, sobre 1.6 millones de bloques de entrenamiento. Inferir un bloque para todas las estaciones toma menos de un segundo, compatible con recalcular la lista al inicio de cada bloque.
- **Advertencia vigente.** Con `class_weight = "balanced"` los scores no son probabilidades calibradas (van den Goorbergh et al., 2022). El modelo sirve para ordenar estaciones; si se quiere comunicar un porcentaje de riesgo, hay que calibrarlo con el periodo de validación.
- **Challenger: Random Forest balanceado** (PR-AUC 0.443 en prueba), por la misma razón que en P1.

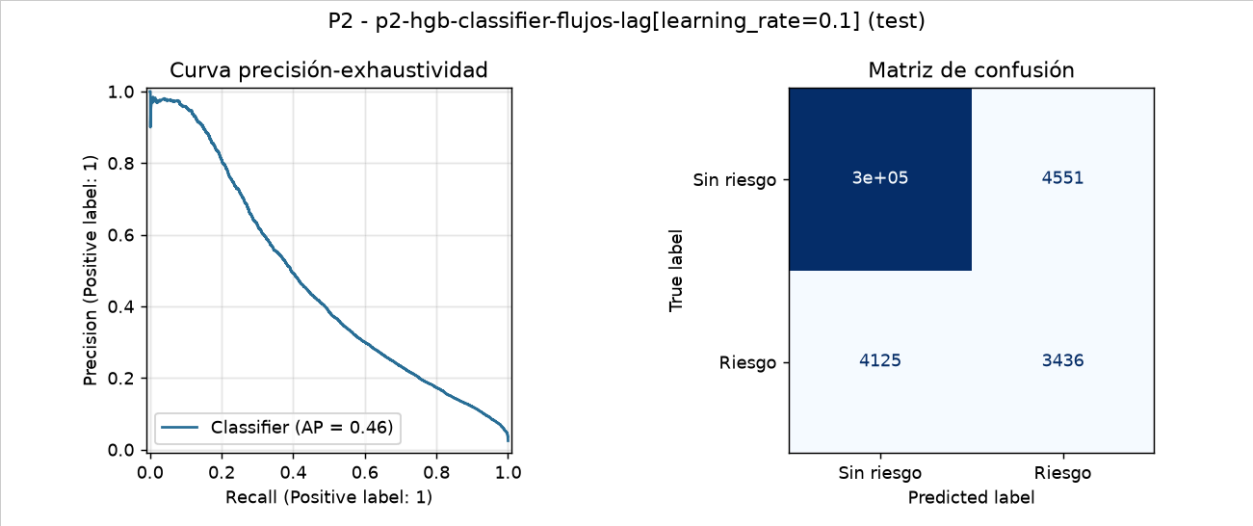



Figura 2. P2, champion en prueba: curva precisión-exhaustividad y matriz de confusión con el umbral de validación (artefacto `plots/`).

### P3: grupos de estaciones por patrón de uso

**Champion: KMeans sobre perfiles de uso con k = 3** (`p3-kmeans-perfiles-uso`). Esta es la pregunta en la que el modelo con la mejor cifra no es el elegido. Por eso el champion se fija

explícitamente en `scripts/run_experiments.sh` en lugar de dejar la selección automática por silueta.

- **Por qué no la mezcla gaussiana.** La mezcla gaussiana con covarianza diagonal tiene la mejor silueta (0.160 frente a 0.142), pero su ARI de estabilidad es 0.305. Con covarianza completa la silueta es 0.155 y el ARI cae a 0.011: con 373 estaciones y 52 variables, cada grupo estima una matriz de 1,378 parámetros, el modelo sobreajusta las estaciones de entrenamiento y la asignación con los perfiles de 2026 es casi aleatoria. Planeación urbana usa los grupos para decidir dónde y con qué perfil de servicio expandir el sistema, y esa decisión no puede depender de grupos que cambian de un año al siguiente.
- **Por qué KMeans y no Birch.** Birch es el más estable (ARI 0.532) y tiene un Davies-Bouldin casi igual (2.05 frente a 2.03), pero su silueta es menor (0.122). KMeans ofrece el mejor equilibrio entre separación y estabilidad, sus centroides se interpretan directamente como perfiles horarios (pico matutino de salidas, pico vespertino y uso recreativo de fin de semana) y es el modelo que el equipo ya validó en la entrega anterior. Birch queda como **challenger**: si en el siguiente periodo la estabilidad del champion baja, es la alternativa natural.
- **Número de grupos.** La búsqueda con nested runs sobre k = 3 a 8 confirma k = 3: la silueta baja de 0.142 a 0.061-0.098 y el ARI de 0.45 a 0.21-0.39 conforme aumenta k. Con k = 8 aparecen grupos de 8 estaciones, demasiado pequeños para planear. MiniBatchKMeans, más barato, pierde separación (0.086), y el ahorro no importa con 373 estaciones: todos los modelos de P3 entrenan en menos de 11 s.
- **Contra los baselines.** El champion mejora la silueta del baseline geográfico en 0.156 (de -0.014 a 0.142) y la de la regla fija de mañana y tarde en 0.091, con un ARI mayor que el de la regla (0.446 frente a 0.271). Una silueta de 0.14 indica grupos con traslape: los perfiles de uso forman un continuo más que tipos separados.

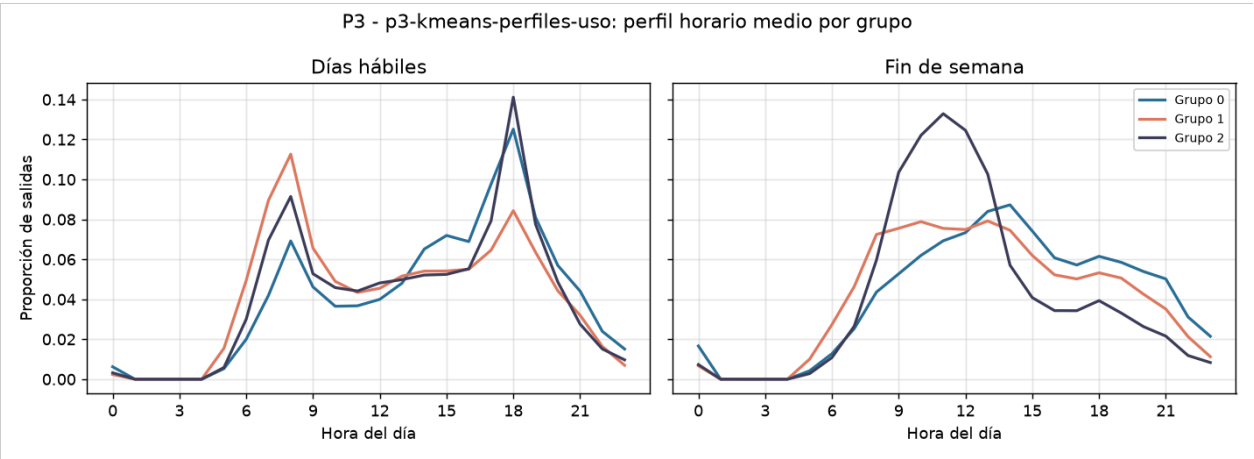



Figura 3. P3, champion: perfil horario medio de salidas de cada grupo (artefacto `plots/`).

## 6. Model Registry y ciclo de vida

El servidor de tracking usa SQLite, que el Model Registry necesita. Cada pregunta tiene un modelo registrado con nombre consistente y una descripción de su tarea, su granularidad y su salida:

Tabla 5. Modelos registrados y versiones en producción.

| Modelo registrado | Descripción | Alias y versión: corrida de origen | Linaje |
|---|---|---|---|
| `mibici-equipo34-p1-demanda` | P1: viajes que salen de cada estación al día siguiente (regresión, estación-día). | **@champion** v1: `p1-hgb-regressor-lags-calendario[learning_rate=0.1]`<br>**@challenger** v2: `p1-random-forest-lags-calendario` | commit `5e3f2c6`<br>data_version `efa3a16661ec`<br>git_dirty false |
| `mibici-equipo34-p2-riesgo` | P2: riesgo de desbalance de una estación en el siguiente bloque de 3 h (clasificación binaria; predict devuelve la etiqueta con el umbral elegido en validación). | **@champion** v1: `p2-hgb-classifier-flujos-lag[learning_rate=0.1]`<br>**@challenger** v2: `p2-random-forest-flujos-lag` | commit `5e3f2c6`<br>data_version `efa3a16661ec`<br>git_dirty false |
| `mibici-equipo34-p3-grupos` | P3: grupo de uso de cada estación según su perfil horario (clustering). | **@champion** v1: `p3-kmeans-perfiles-uso`<br>**@challenger** v2: `p3-birch-perfiles-uso` | commit `5e3f2c6`<br>data_version `efa3a16661ec`<br>git_dirty false |

**Linaje.** Cada versión guarda como etiquetas el experimento, la corrida de origen (`source_run_id` y `source_run_name`), el commit (`git_commit`, con `git_dirty = false`), la versión de los datos (`data_version`), el hash del conjunto de entrenamiento (`train_data_hash`), la métrica de selección con su valor y, si la versión viene de una búsqueda, la corrida padre (`search_parent_run_id`). Desde cualquier versión se llega a los parámetros, las métricas y los artefactos de la corrida que la produjo, y desde ahí al código exacto y a los datos.

**Esquema: aliases en lugar de etapas.** El equipo usa aliases: `champion` para la versión en producción y `challenger` para la siguiente mejor alternativa. Las etapas clásicas (`Staging`, `Production`, `Archived`) están obsoletas desde MLflow 2.9 y la documentación vigente recomienda aliases (MLflow, s.f.-a). Es la práctica que describen Treveil et al. (2020) para separar el ciclo de vida del modelo del código que lo consume. Además, los aliases resuelven tres necesidades del proyecto:

- **Desacoplan el código del número de versión.** `predict.py` y las siguientes etapas cargan `models:/mibici-equipo34-p1-demanda@champion`. Promover o revertir un modelo es mover el alias, sin cambiar código ni volver a desplegar.
- **Permiten tener varias versiones activas con nombre propio.** Una etapa solo admite un estado por versión. Con aliases conviven el `champion` y el `challenger`, y más adelante se pueden agregar otros (por ejemplo, `shadow` para evaluarlo en paralelo con datos nuevos).
- **Hacen auditable la decisión.** La promoción está en `scripts/run_experiments.sh` (selección por métrica en P1 y P2, run fijado y comentado en P3), así que el estado del registry se

reconstruye con el mismo comando que el resto del experimento.

**Verificación.** Al asignar el alias, el punto de entrada `register` carga `models:/<nombre>@<alias>` con `mlflow.pyfunc.load_model` y predice el ejemplo de entrada guardado con el modelo; si algo falla, el registro falla. Las seis cargas (tres `champion` y tres `challenger`) fueron exitosas en la corrida oficial. `predict.py` (`make predict Q=p1`) usa ese mismo alias para generar las predicciones de prueba con el champion.

## 7. Flujo de trabajo con Git y GitHub

La rama `main` está protegida: todo cambio entra por Pull Request revisado. Las ramas siguen `<tipo>/<descripción>` y los commits, Conventional Commits.

Tabla 6. Pull Requests de esta entrega.

| PR | Autor | Contenido | Revisión |
|---|---|---|---|
| #13 | Oscar Elton | Servidor de tracking, `MLproject` y `python_env.yaml`; módulos por pregunta; artefactos en subcarpetas; búsquedas con nested runs; Model Registry con aliases; `run_experiments.sh`. | Diego Falcón (aprobado) |
| #14 | Javier García | Integrantes y matrículas en la propuesta. | Mariana Favarony (aprobado) |
| #15 | Javier García | Variantes RandomForest, Ridge, MiniBatchKMeans, Birch y mezcla gaussiana. | Mariana Favarony (aprobado) |
| #16 | Mariana Favarony | Búsquedas de P1, P2 y P3 en el script de experimentos y primera versión del análisis. | Oscar Elton (aprobado) |
| #17 | Diego Falcón | Integración de las variantes del PR #15 al MLflow Project, corrección para Windows, champion y challenger justificados, corrida oficial, README y este reporte. | Javier García (revisión solicitada) |

## Referencias


Lakshmanan, V., Robinson, S., & Munn, M. (2020). *Machine learning design patterns: Solutions to common challenges in data preparation, model building, and MLOps*. O'Reilly Media.

MiBici. (s.f.). *Datos abiertos* [Conjunto de datos]. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. https://www.mibici.net/es/datos-abiertos/

MLflow. (s.f.-a). *MLflow Model Registry* [Documentación]. MLflow. https://mlflow.org/docs/latest/ml/model-registry/

MLflow. (s.f.-b). *MLflow Projects* [Documentación]. MLflow. https://mlflow.org/docs/latest/ml/projects/

MLflow. (s.f.-c). *MLflow Tracking* [Documentación]. MLflow. https://mlflow.org/docs/latest/ml/tracking/

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53-65. https://doi.org/10.1016/0377-0427(87)90125-7

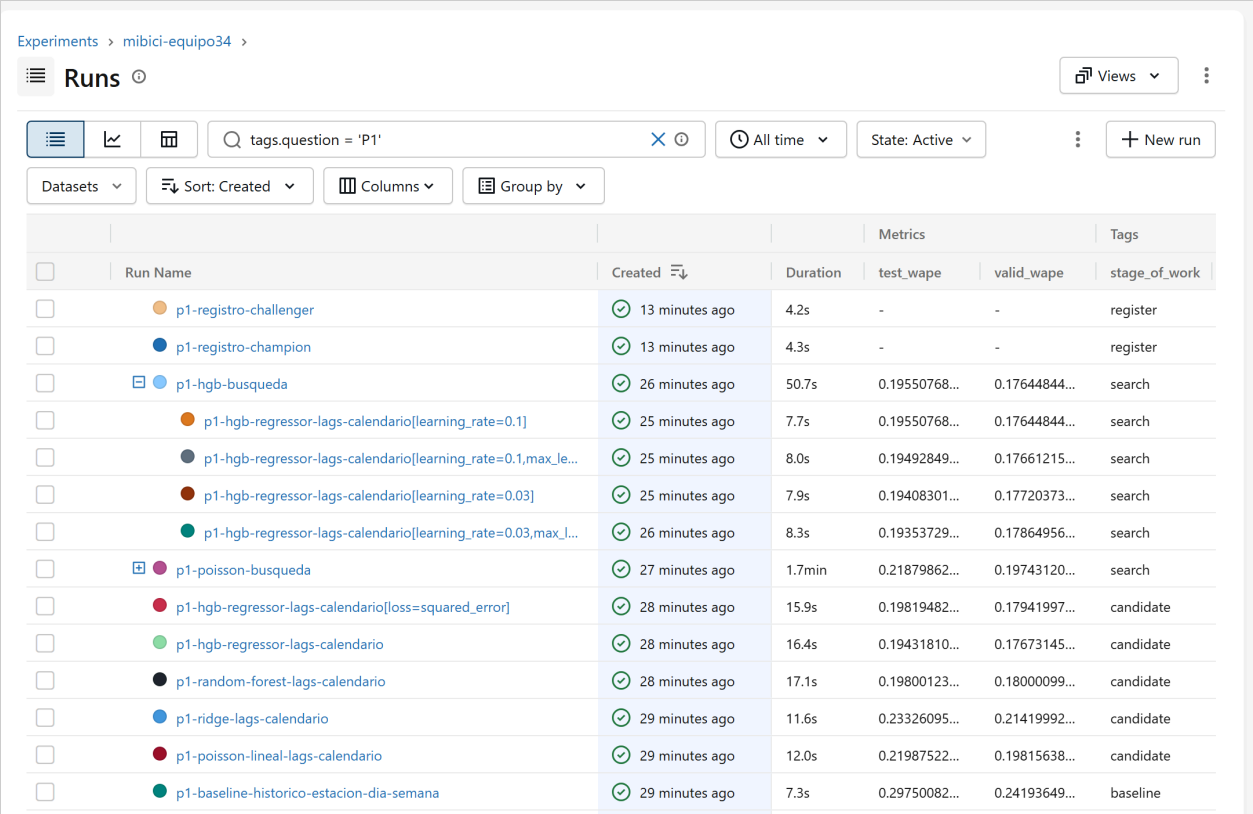
Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE, 10*(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). *Introducing MLOps: How to scale machine learning in the enterprise*. O'Reilly Media.

van den Goorbergh, R., van Smeden, M., Timmerman, D., & Van Calster, B. (2022). The harm of class imbalance corrections for risk prediction models: Illustration and simulation using logistic regression. *Journal of the American Medical Informatics Association, 29*(9), 1525-1534. https://doi.org/10.1093/jamia/ocac093

## Anexo: evidencia de la interfaz de MLflow

Capturas del servidor de tracking de la corrida oficial (experimento `mibici-equipo34`, 64 corridas lanzadas con `mlflow run` desde un clon limpio).




| Run Name | Created | Duration | Metrics: test_wape | Metrics: valid_wape | Tags: stage_of_work |
|---|---|---|---|---|---|
| p1-registro-challenger | 13 minutes ago | 4.2s | - | - | register |
| p1-registro-champion | 13 minutes ago | 4.3s | - | - | register |
| p1-hgb-busqueda | 26 minutes ago | 50.7s | 0.19550768... | 0.17644844... | search |
| p1-hgb-regressor-lags-calendario[learning_rate=0.1] | 25 minutes ago | 7.7s | 0.19550768... | 0.17644844... | search |
| p1-hgb-regressor-lags-calendario[learning_rate=0.1,max_le... | 25 minutes ago | 8.0s | 0.19492849... | 0.17661215... | search |
| p1-hgb-regressor-lags-calendario[learning_rate=0.03] | 25 minutes ago | 7.9s | 0.19408301... | 0.17720373... | search |
| p1-hgb-regressor-lags-calendario[learning_rate=0.03,max_l... | 26 minutes ago | 8.3s | 0.19353729... | 0.17864956... | search |
| p1-poisson-busqueda | 27 minutes ago | 1.7min | 0.21879862... | 0.19743120... | search |
| p1-hgb-regressor-lags-calendario[loss=squared_error] | 28 minutes ago | 15.9s | 0.19819482... | 0.17941997... | candidate |
| p1-hgb-regressor-lags-calendario | 28 minutes ago | 16.4s | 0.19431810... | 0.17673145... | candidate |
| p1-random-forest-lags-calendario | 28 minutes ago | 17.1s | 0.19800123... | 0.18000099... | candidate |
| p1-ridge-lags-calendario | 29 minutes ago | 11.6s | 0.23326095... | 0.21419992... | candidate |
| p1-poisson-lineal-lags-calendario | 29 minutes ago | 12.0s | 0.21987522... | 0.19815638... | candidate |
| p1-baseline-historico-estacion-dia-semana | 29 minutes ago | 7.3s | 0.29750082... | 0.24193649... | baseline |

Figura A1. Lista de corridas de P1 con la búsqueda `p1-hgb-busqueda` expandida: la corrida padre y sus cuatro corridas hijas (nested runs), junto a los baselines y las candidatas, con las métricas de validación y prueba como columnas.

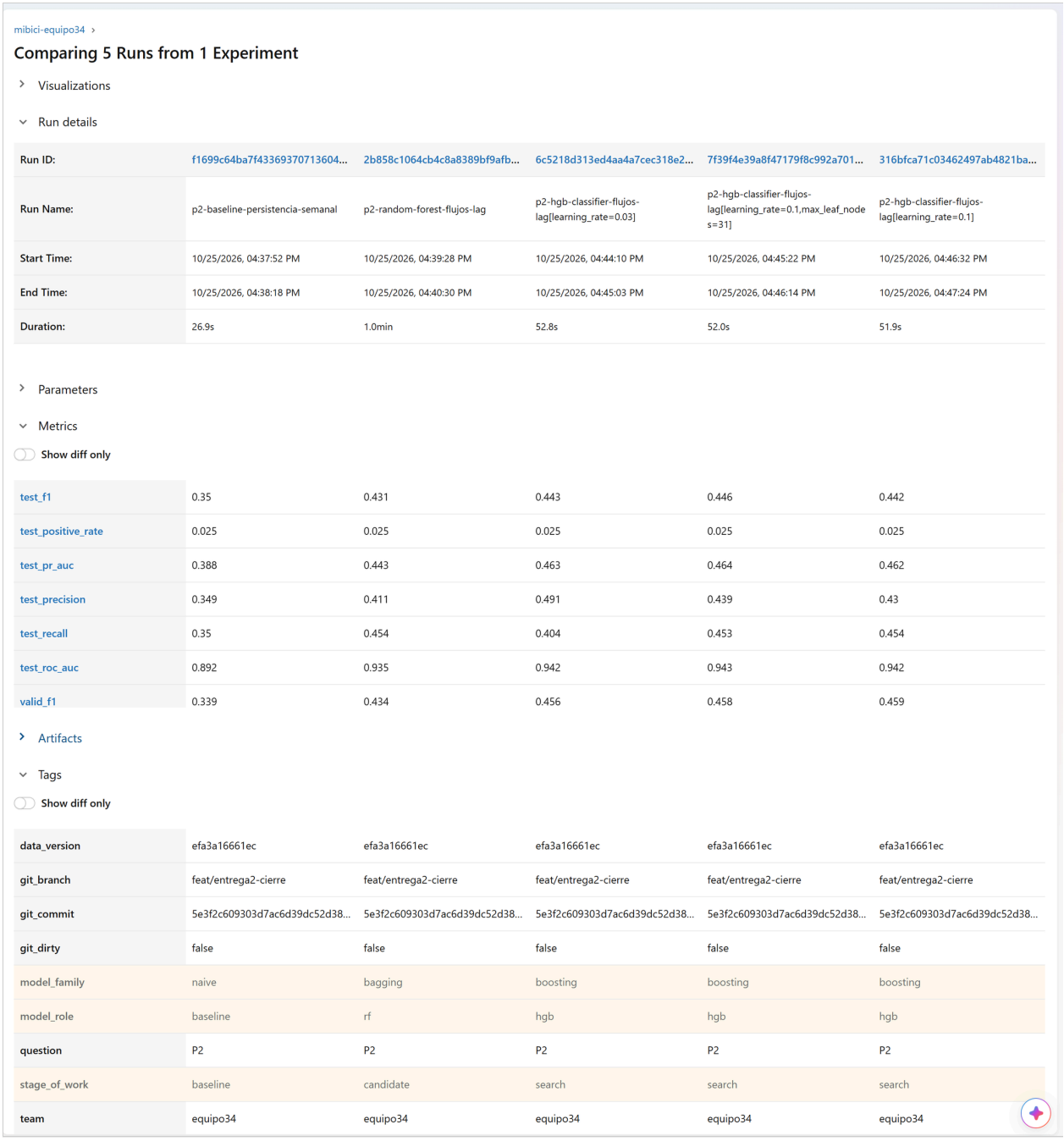



Figura A2. Comparación en la interfaz de tres corridas hijas de `p2-hgb-busqueda` (la última es el champion) contra el baseline de persistencia semanal y el Random Forest: duración, métricas de prueba y etiquetas lado a lado (sección de parámetros colapsada).

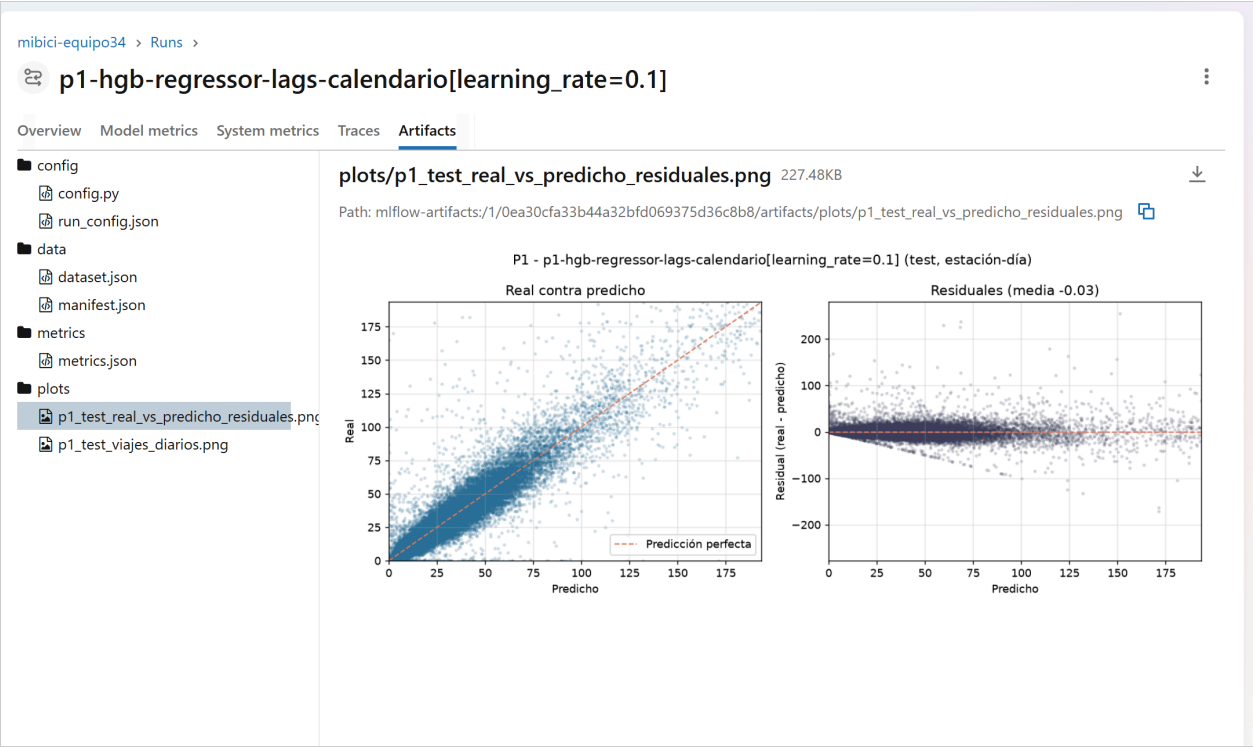



Figura A3. Artefactos de la corrida candidata `p1-hgb-regressor-lags-calendario[learning_rate=0.1]`, organizados en subcarpetas: `config/` (copia de `config.py` e hiperparámetros resueltos), `data/` (manifiesto de la ingesta y huella del conjunto de entrenamiento), `metrics/` y `plots/`.

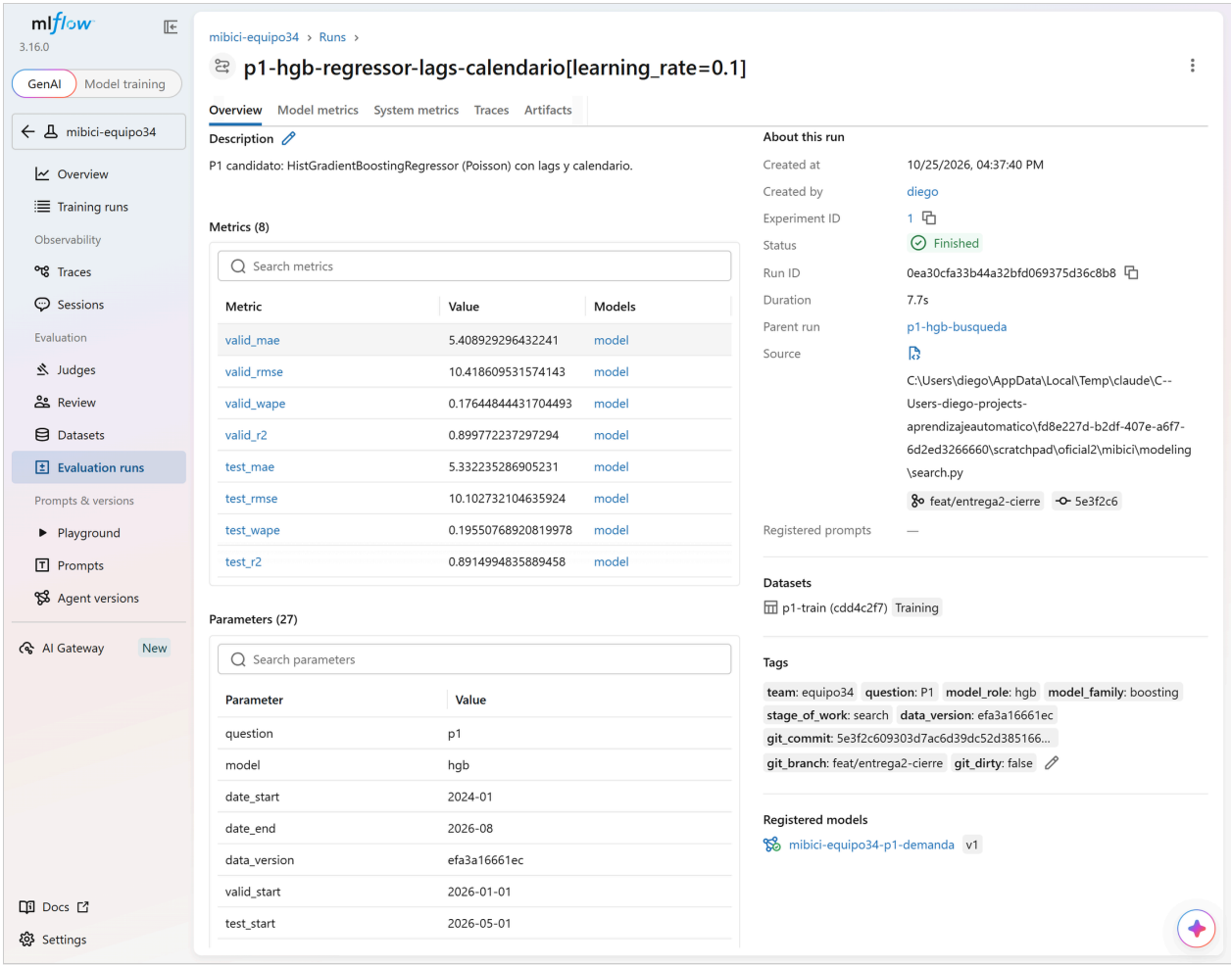



Figura A3b. Detalle de la misma corrida: métricas ligadas al modelo, corrida padre de la búsqueda, conjunto de entrenamiento registrado con `log_input` (`p1-train`), commit y rama de origen, y etiquetas (`question`, `model_family`, `stage_of_work`, `data_version`, `git_commit`, `git_dirty = false`).

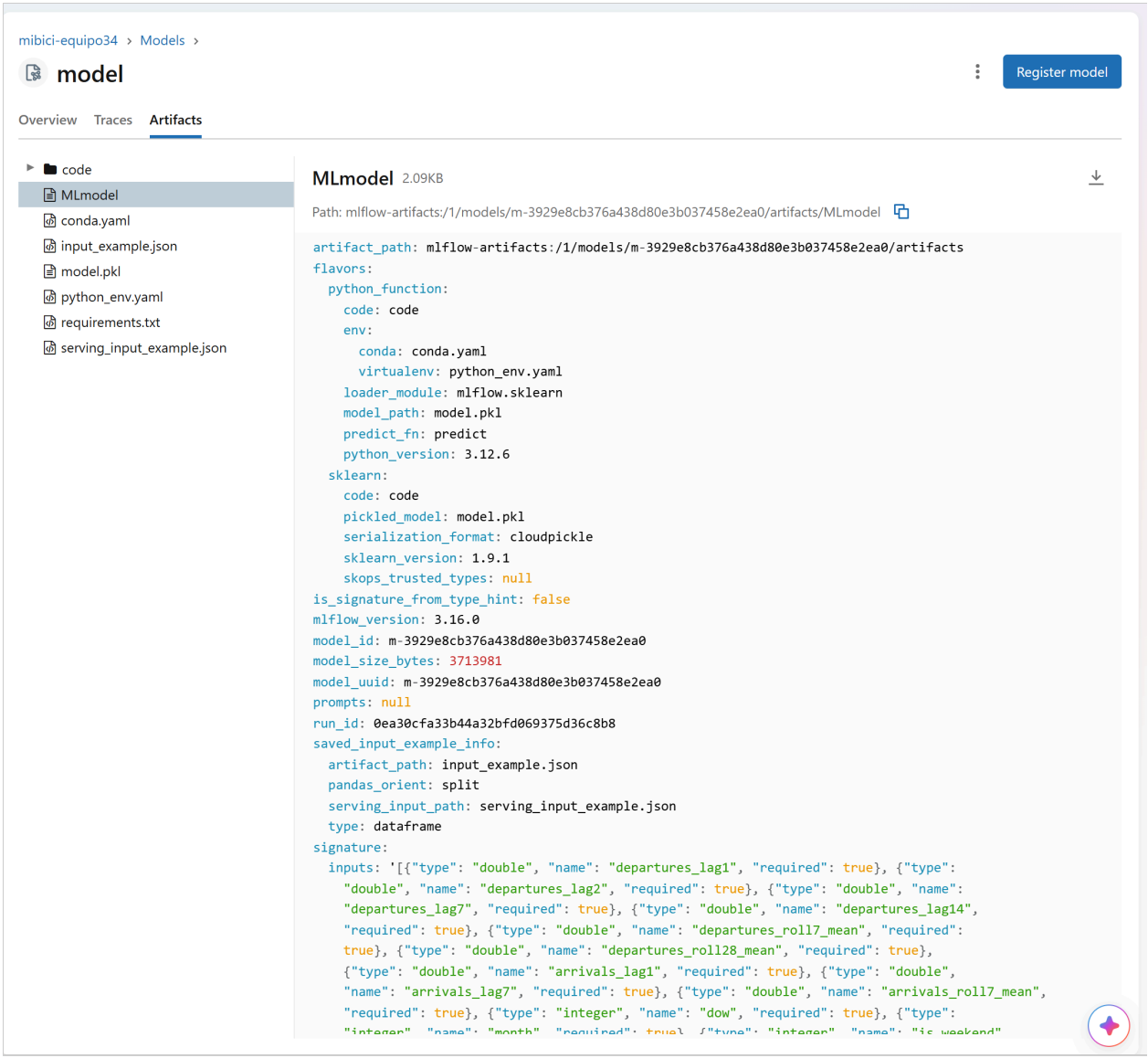



Figura A3c. Modelo de la corrida (en MLflow 3 se guarda como *Logged Model* ligado a la corrida): archivo `MLmodel` con la firma (`signature`) y el ejemplo de entrada (`input_example.json`).

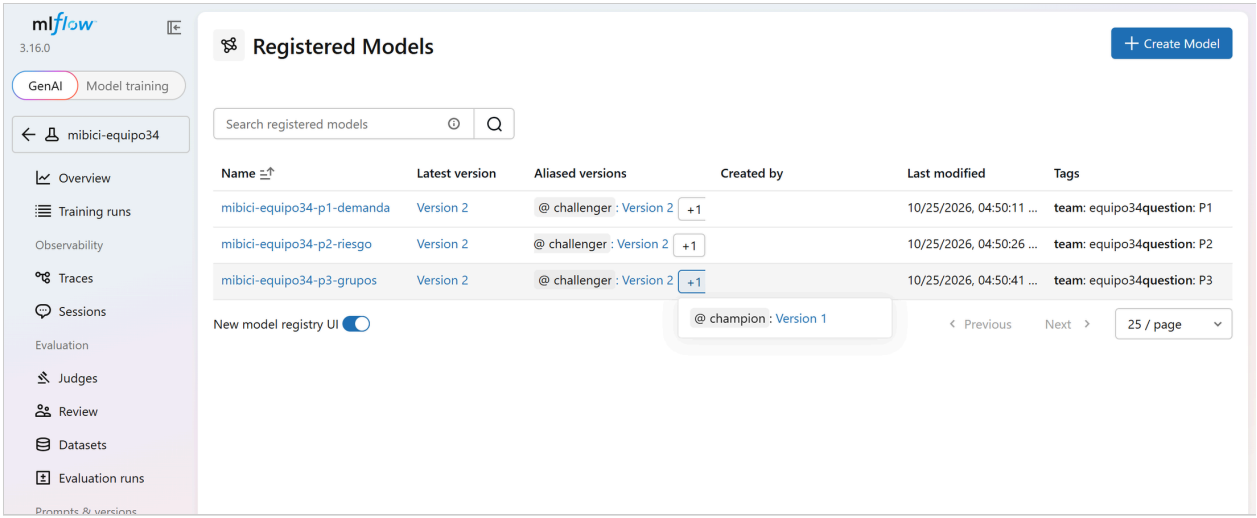



Figura A4. Model Registry: los tres modelos registrados. La columna de aliases muestra `@challenger` (versión 2) y, al desplegar "+1", `@champion` (versión 1), como se ve en la tercera fila.

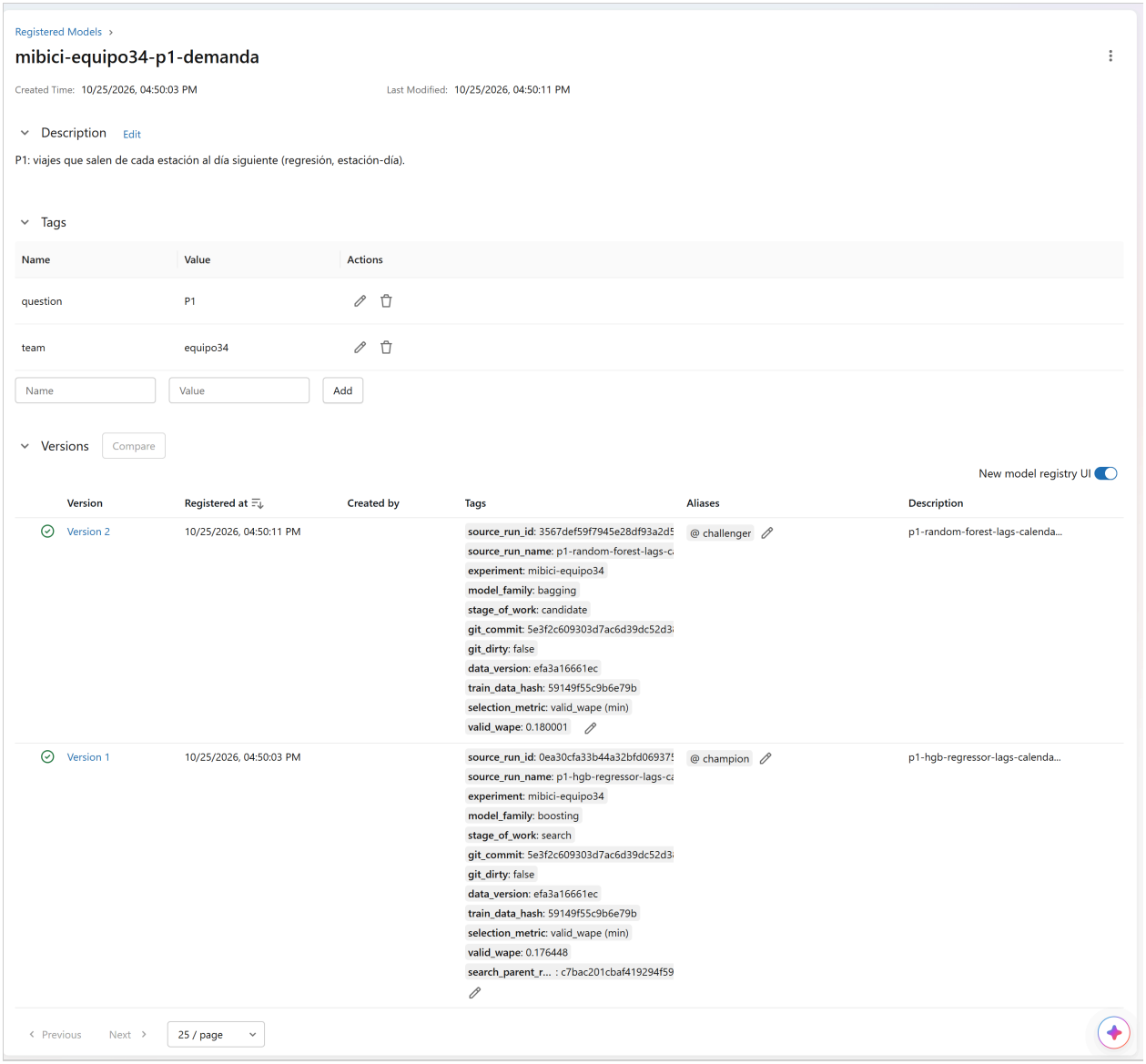



Figura A5. `mibici-equipo34-p1-demanda`: descripción del modelo y sus versiones con alias y etiquetas de linaje (corrida de origen, experimento, familia, etapa de trabajo, commit, `git_dirty`, `data_version`, hash del conjunto de entrenamiento, métrica de selección y búsqueda padre). La versión 1 es `@champion`.

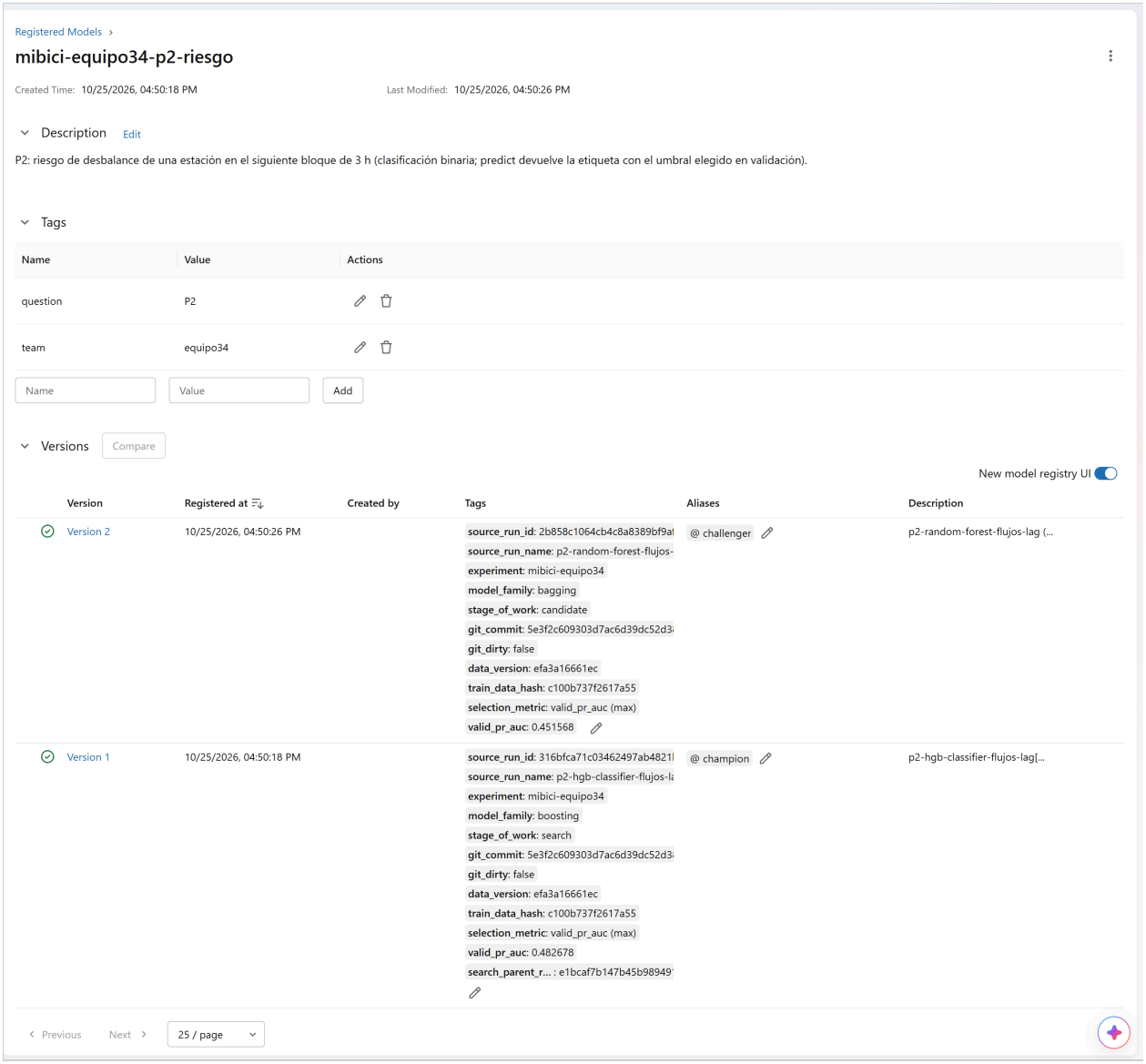



Figura A6. `mibici-equipo34-p2-riesgo`: versión 1 (`@champion`, HGB de la búsqueda) y versión 2 (`@challenger`, Random Forest), con su linaje.

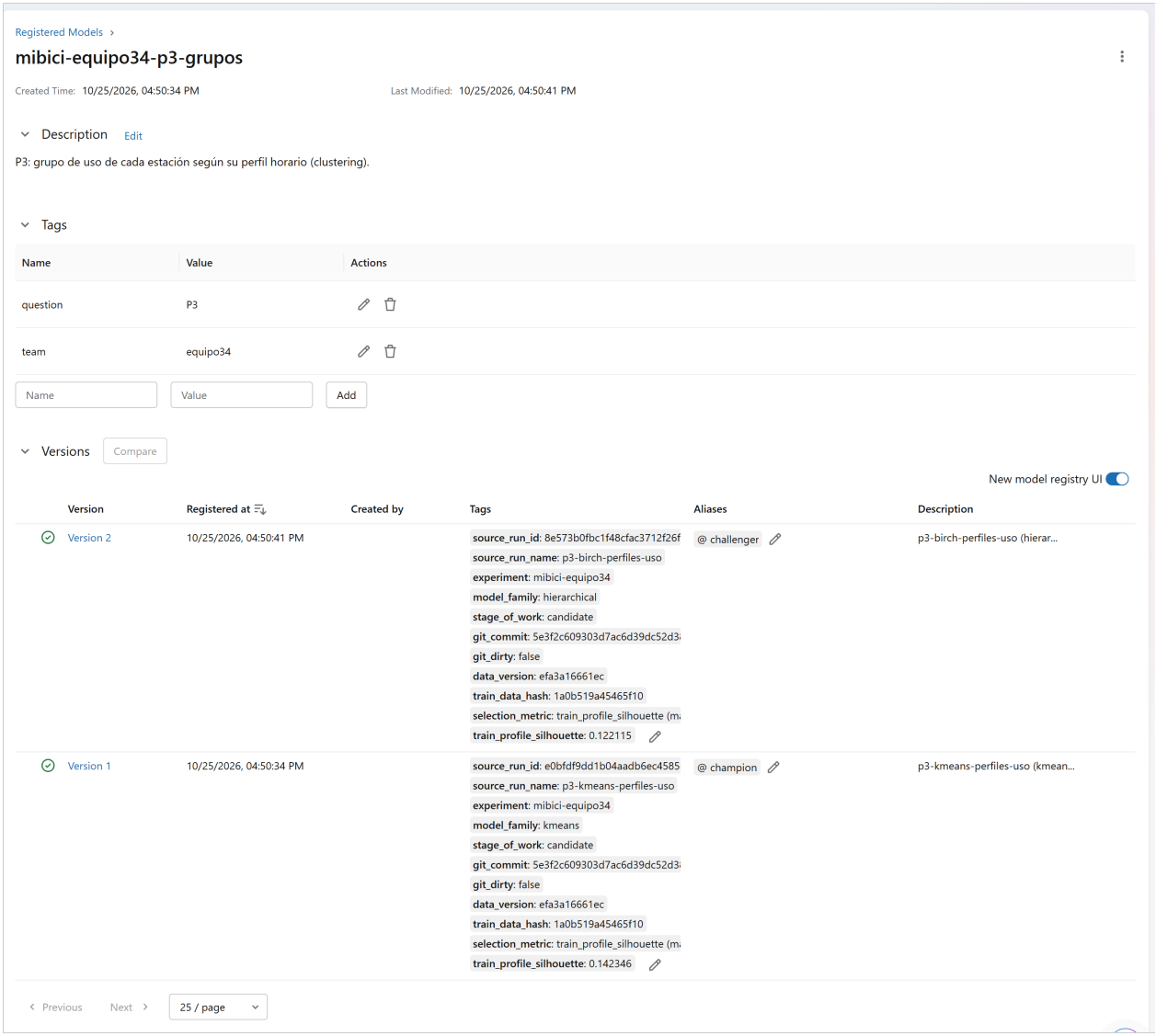



Figura A7. `mibici-equipo34-p3-grupos`: versión 1 (`@champion`, KMeans k = 3, fijada en `scripts/run_experiments.sh`) y versión 2 (`@challenger`, Birch), con su linaje.